WALKTHROUGH REPORT

Vulnerable Machine: RickdiculouslyEasy

Penetration Testing Documentation — Personal Portfolio

Prepared by: Reinaldy

Target Machine	RickdiculouslyEasy (Boot2Root VM)
Target IP Address	`192.168.56.104`
Attacking Machine	Kali Linux
Main Tools	nmap, ftp, netcat, nikto, dirb, hydra, python, ssh, scp
Total Flags / Points	9 Flags — 130 Points

Summary

RickdiculouslyEasy is a boot-to-root virtual machine with a “Rick and Morty” theme, designed to practice service enumeration, command injection exploitation, credential brute-forcing, and privilege escalation to root. Testing was carried out entirely within an isolated lab environment (virtual network 192.168.56.0/24) using Kali Linux as the attacking machine.

During the assessment, all 9 flags were captured for a total of 130 points, through the following stages:

- Enumeration of open services (FTP, SSH, HTTP, Cockpit) using Nmap.
- Capture of early flags via anonymous FTP access and netcat connections to non-standard ports (backdoor/reverse shell).
- Content discovery on the web service (Nikto & Dirb), revealing hidden directories and a robots.txt file.
- Exploitation of an OS command injection vulnerability in a CGI script (tracertool.cgi) to read system files (/etc/passwd).
- Analysis of credentials stored in an HTML comment, leading to SSH access as a standard user.
- Extraction of hidden information from an image file (simple steganography via head/tail) to unlock a password-protected archive.
- Creation of a custom wordlist with Python based on a password hint, followed by an SSH brute-force attack using Hydra.
- Privilege escalation to root via a SUID binary/sudo access to retrieve the final flag.

Reconnaissance

The initial stage involved a full TCP port scan of the target using the command:

```
nmap -sCV -p- 192.168.56.104 -oA output
```

The scan revealed seven open services:

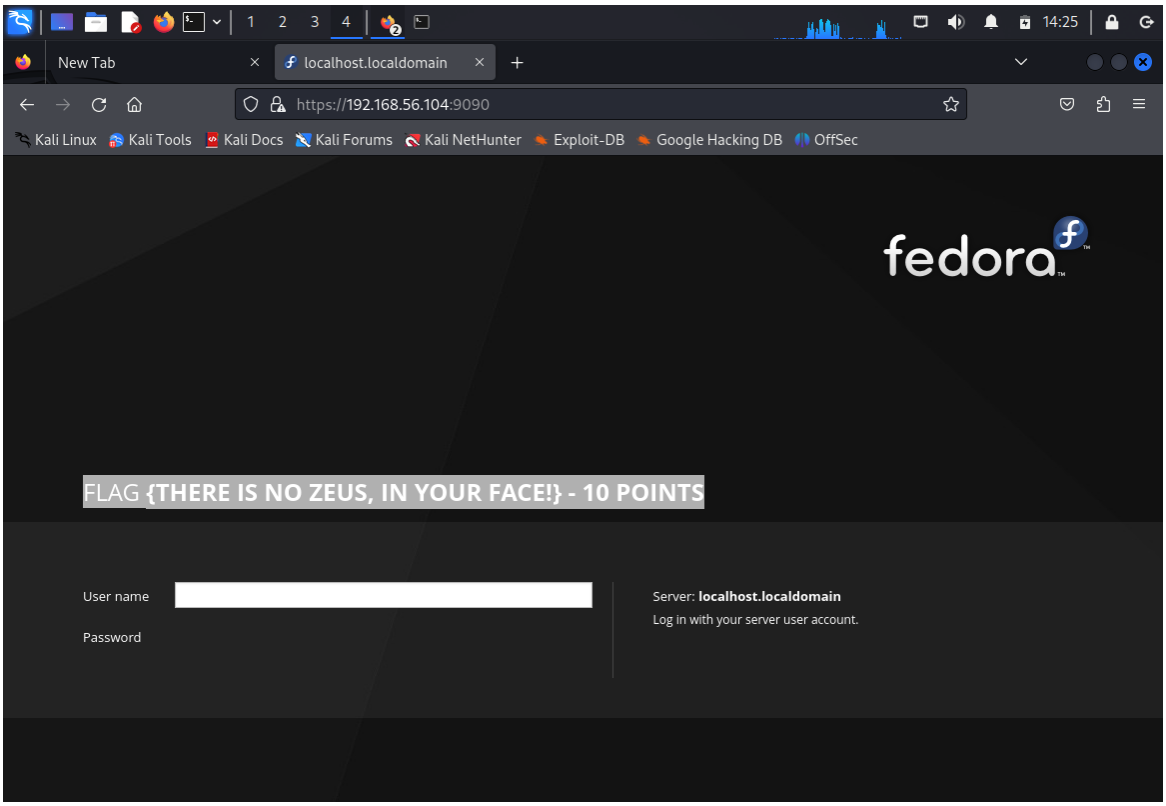
Port	Service	Version	Notes
21/tcp	FTP	vsftpd 3.0.3	Allows Anonymous login
22/tcp	SSH	OpenSSH (fingerprint failed)	Likely a decoy service; the real SSH server runs on 22222/tcp
80/tcp	HTTP	Apache 2.4.27 (Fedora)	Page title: “Morty's Website”
9090/tcp	HTTP (Cockpit)	Cockpit web service	Redirects to HTTPS, server login page
13337/tcp	Unknown	—	NULL probe fingerprint directly returns FLAG.txt (backdoor)
22222/tcp	SSH	OpenSSH 7.5 (protocol 2.0)	The actual SSH server used for user login
60000/tcp	Unknown	—	NULL/ibm-db2 probe fingerprint triggers “Rick's half baked reverse shell”

Ports 13337 and 60000 were identified through Nmap's version/fingerprint scan (not merely random connection attempts) — both respond directly to a NULL probe with a banner pointing to a flag/backdoor, which was then confirmed manually via netcat to retrieve the flag. The standard port 22 appears to run SSH but fails hostkey fingerprinting, while the SSH service actually used for authentication runs on port 22222.

Flag 1 — Cockpit Login Page (10 Points)

Finding

Accessing the service on port 9090 via browser (https://192.168.56.104:9090) displays a Cockpit login page. The flag is displayed directly on the page itself, with no further authentication required.



The flag is shown directly on the Cockpit login page (port 9090).

FLAG `{There is no Zeus, in your face!}` — *10 Points*

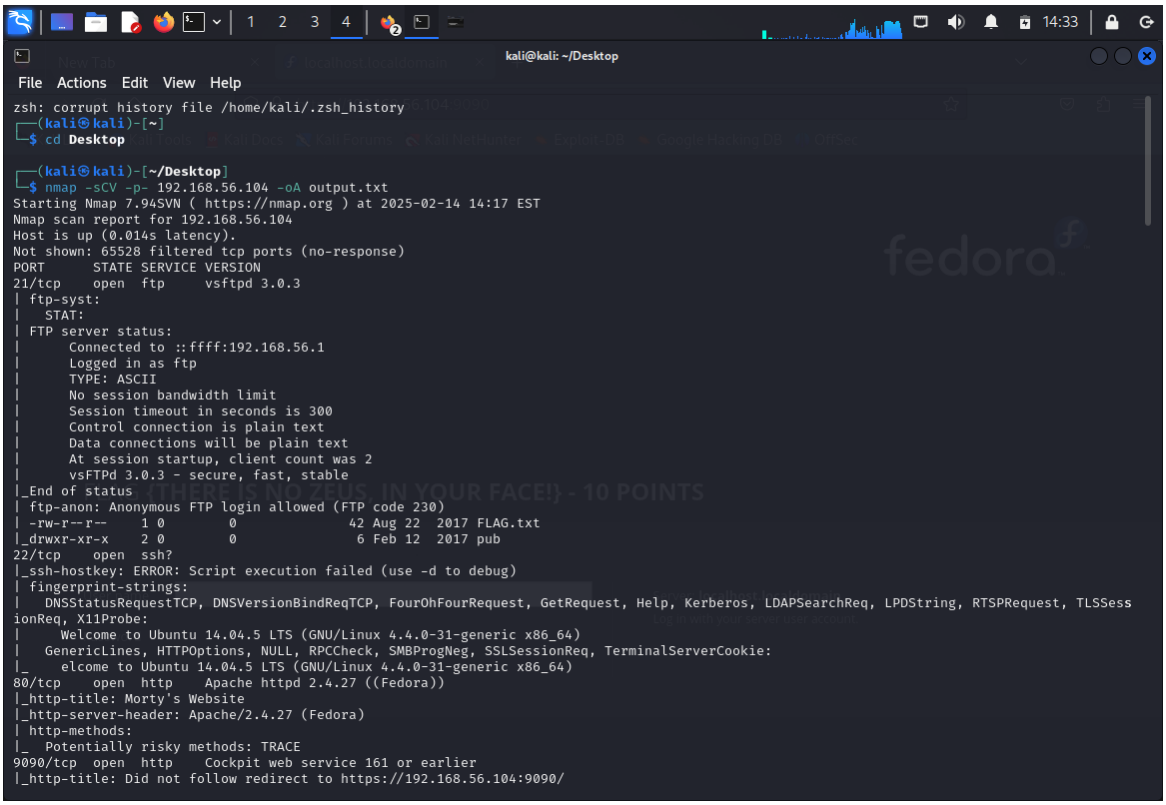
Flag 2 — FTP Anonymous Login (10 Points)

Finding

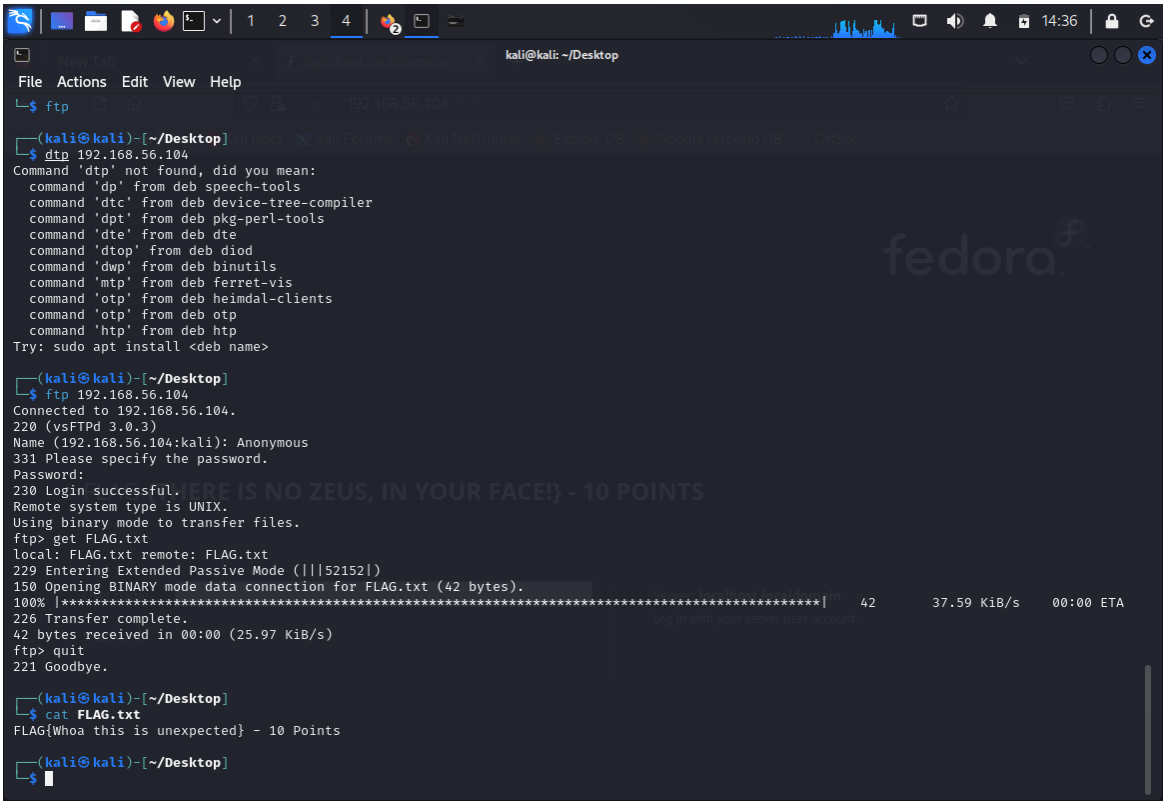
The Nmap scan showed that port 21 (FTP) allows Anonymous login and hosts a directly accessible FLAG.txt file.

Steps Taken

- Logged into the FTP service using anonymous credentials.
- Downloaded FLAG.txt using the get command.
- Read the file contents using cat.



Nmap scan results showing anonymous FTP login is allowed.



Anonymous FTP login, downloading, and reading FLAG.txt.

FLAG {Whoa this is unexpected} — *10 Points*

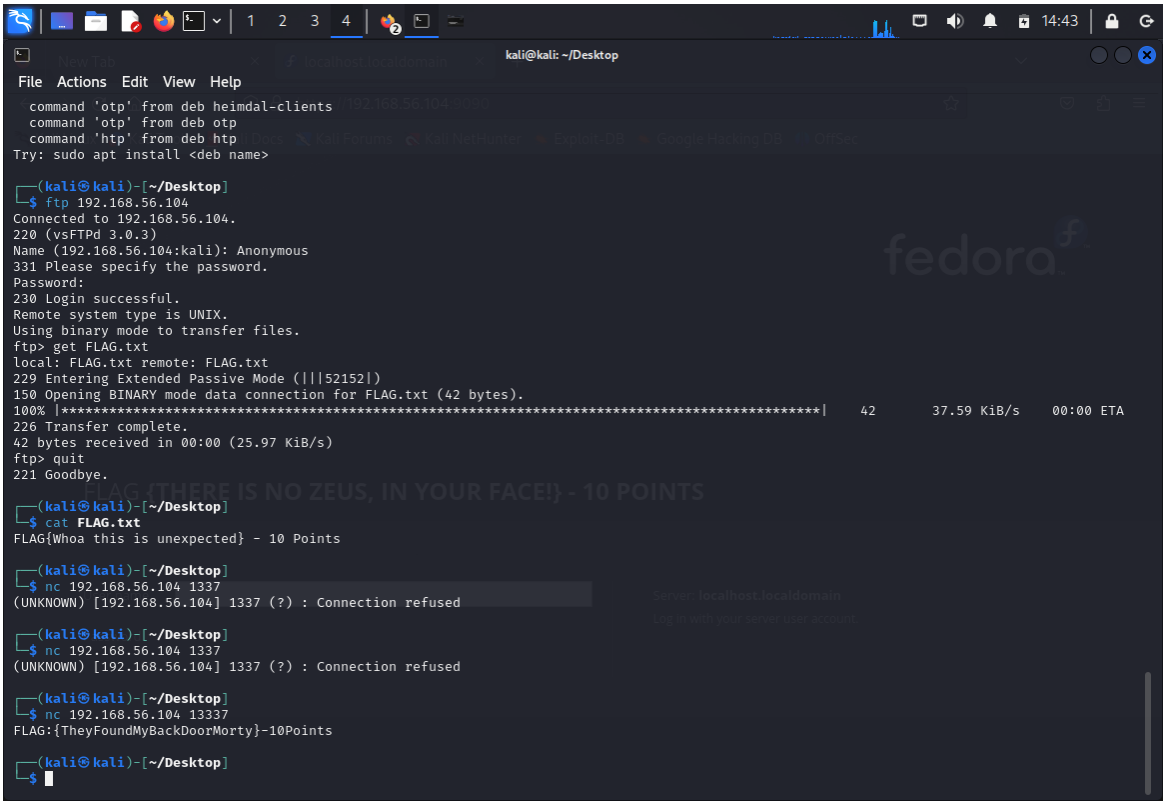
Flag 3 — Netcat Backdoor on Port 13337 (10 Points)

Finding

Nmap's version scan (fingerprint-strings) showed that port 13337/tcp responds to a NULL probe with a string that directly contains the flag. This was confirmed by manually connecting via netcat to that port.

Steps Taken

- An initial attempt on port 1337 failed (connection refused).
- A successful connection was made to port 13337 using: nc 192.168.56.104 13337.



A netcat connection to port 13337 directly returns the flag.

FLAG `{TheyFoundMyBackDoorMorty}` — *10 Points*

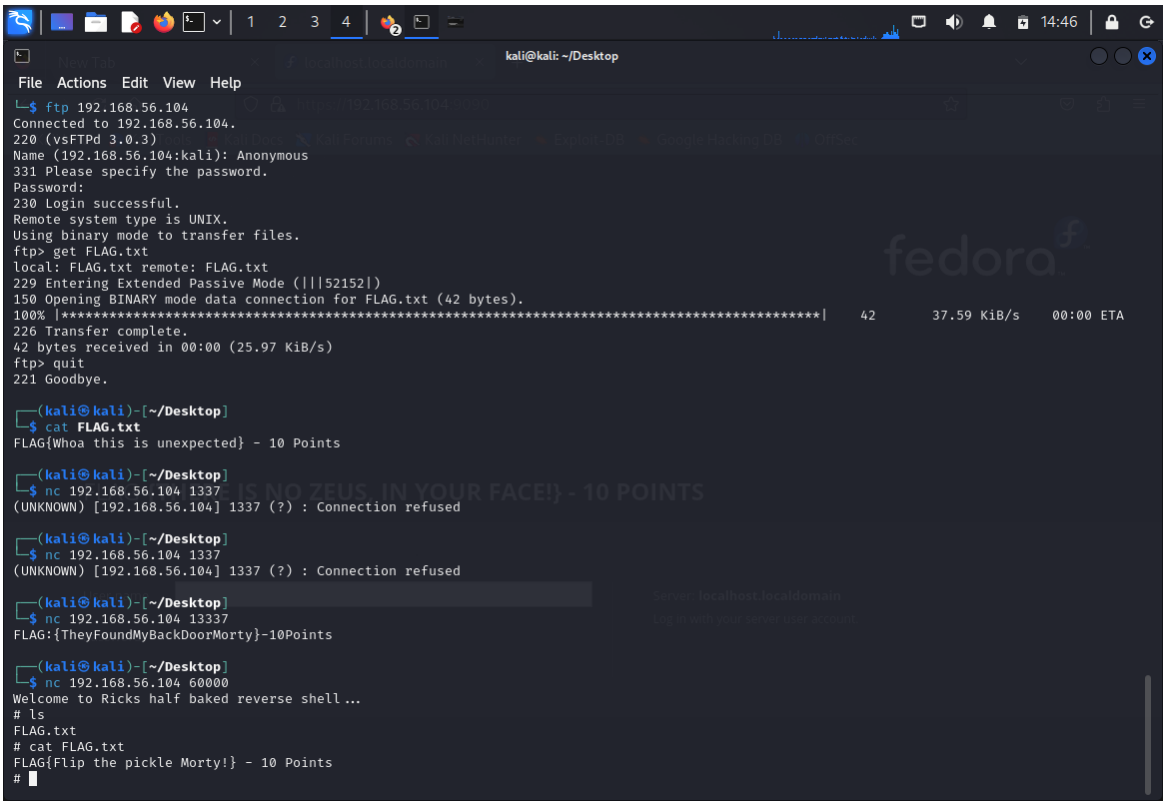
Flag 4 — Reverse Shell on Port 60000 (10 Points)

Finding

Nmap's version scan also showed that port 60000/tcp responds to a NULL/ibm-db2 probe with the banner “Welcome to Rick's half baked reverse shell” — an interactive shell accessible directly with no authentication.

Steps Taken

- Connected via netcat to port 60000: nc 192.168.56.104 60000.
- Once the shell was open, ran ls and cat FLAG.txt to read the flag.



The shell obtained from port 60000 and the contents of FLAG.txt.

FLAG **{Flip the pickle Morty!}** — *10 Points*

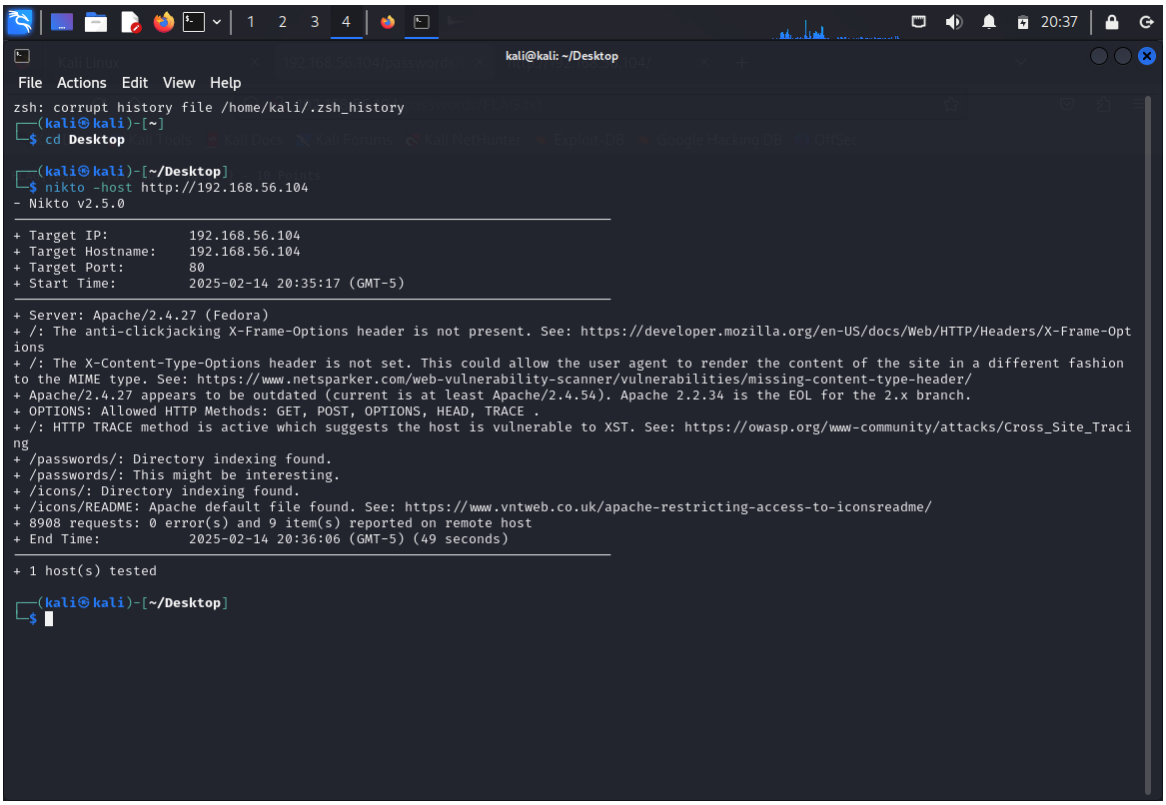
Flag 5 — Indexed /passwords Directory (10 Points)

Finding

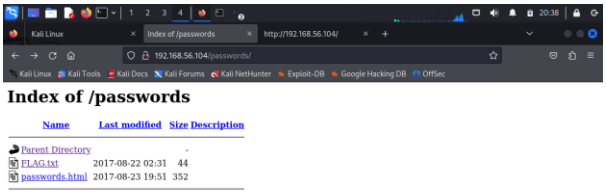
A Nikto scan against the web service (port 80) found that the /passwords/ directory has directory listing enabled and contains a FLAG.txt file.

Steps Taken

- Ran nikto -host http://192.168.56.104.
- Visited http://192.168.56.104/passwords/ in the browser and found the directory was listable.
- Retrieved FLAG.txt from that directory.



Nikto scan results showing the /passwords/ directory with indexing enabled.



Contents of the /passwords/ directory showing FLAG.txt and passwords.html.

FLAG `{Yeah, don't do it.}` — *10 Points*

Flag 6 — Command Injection in a CGI Script (10 Points)

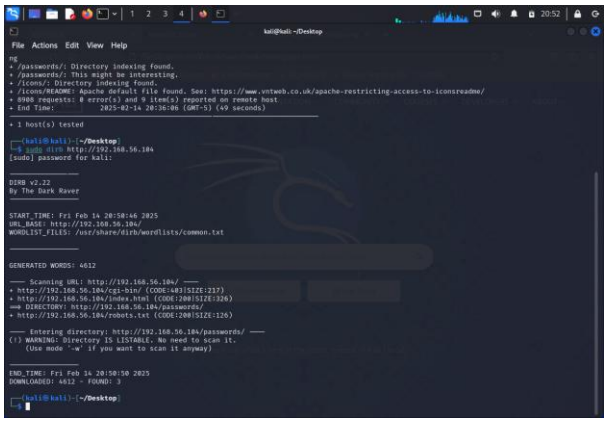
Finding

A directory scan with Dirb found a robots.txt file that inadvertently leaked two CGI endpoints: /cgi-bin/root_shell.cgi (still under construction) and /cgi-bin/tracertool.cgi. The latter is vulnerable to OS command injection: input in the IP address field is executed directly by the shell without sanitization, so the ";" character can be used to inject additional commands.

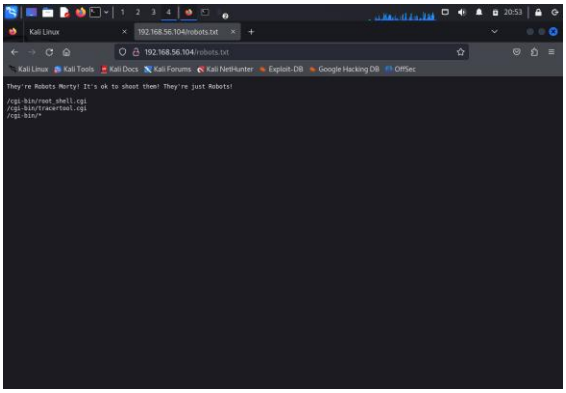
Steps Taken

- Ran sudo dirb http://192.168.56.104 for directory enumeration, finding /passwords/ and robots.txt.

- Read robots.txt, which revealed the /cgi-bin/tracertool.cgi endpoint.
- Injected a command via the ip parameter, e.g. tracertool.cgi?ip=;ls -la /etc/, to read the contents of the /etc directory.
- Read /etc/passwd using ;tail /etc/passwd, revealing three unique accounts: RickSanchez, Morty, and Summer.
- Opened passwords.html (previously found in Flag 5) — the credential “winter” was found hidden inside an HTML comment.
- Since the password “winter” was most contextually relevant to the user Summer, an SSH login attempt on port 22222 was made with user Summer, and it succeeded.



Dirb results finding the /passwords/ directory and robots.txt.



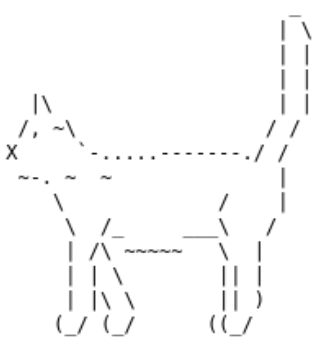
Contents of robots.txt leaking the /cgi-bin/tracertool.cgi endpoint.

MORTY'S MACHINE TRACER MACHINE

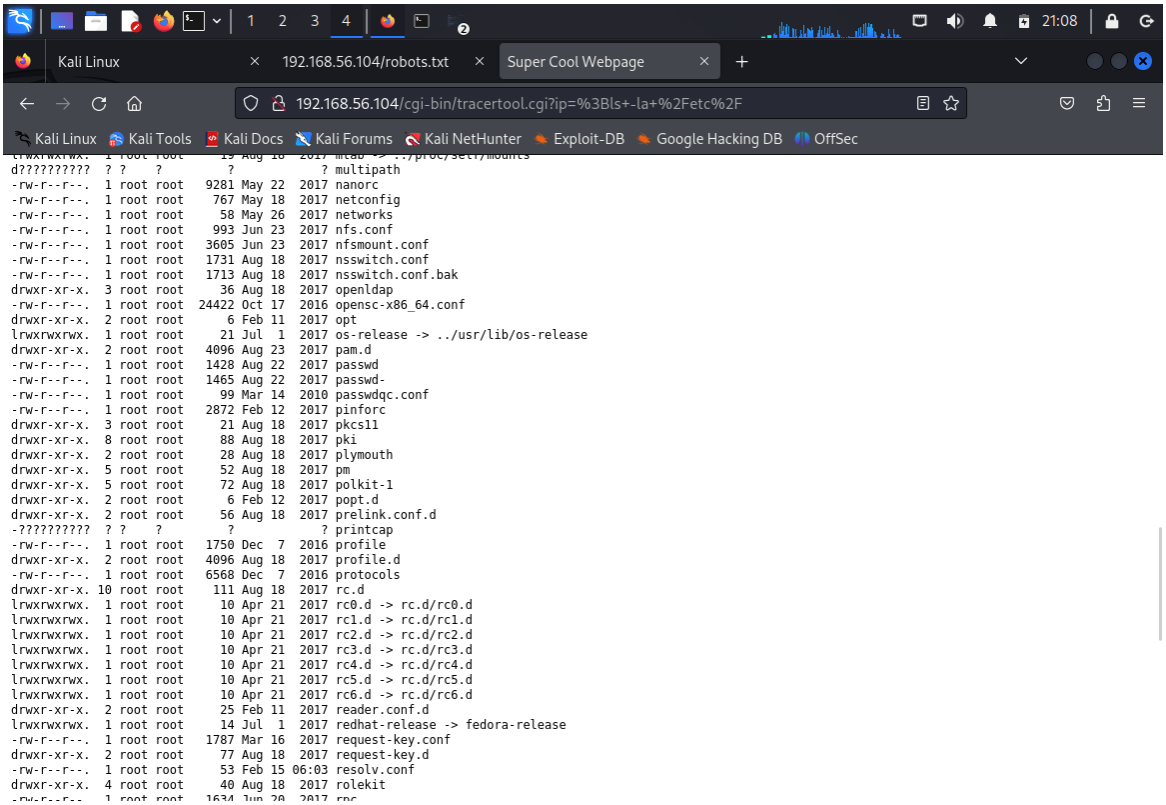
Enter an IP address to trace.

```
;cat
```

Trace!



The tracertool.cgi form is vulnerable to OS command injection.



Command injection ;ls -la /etc/ proves the vulnerability is exploitable.

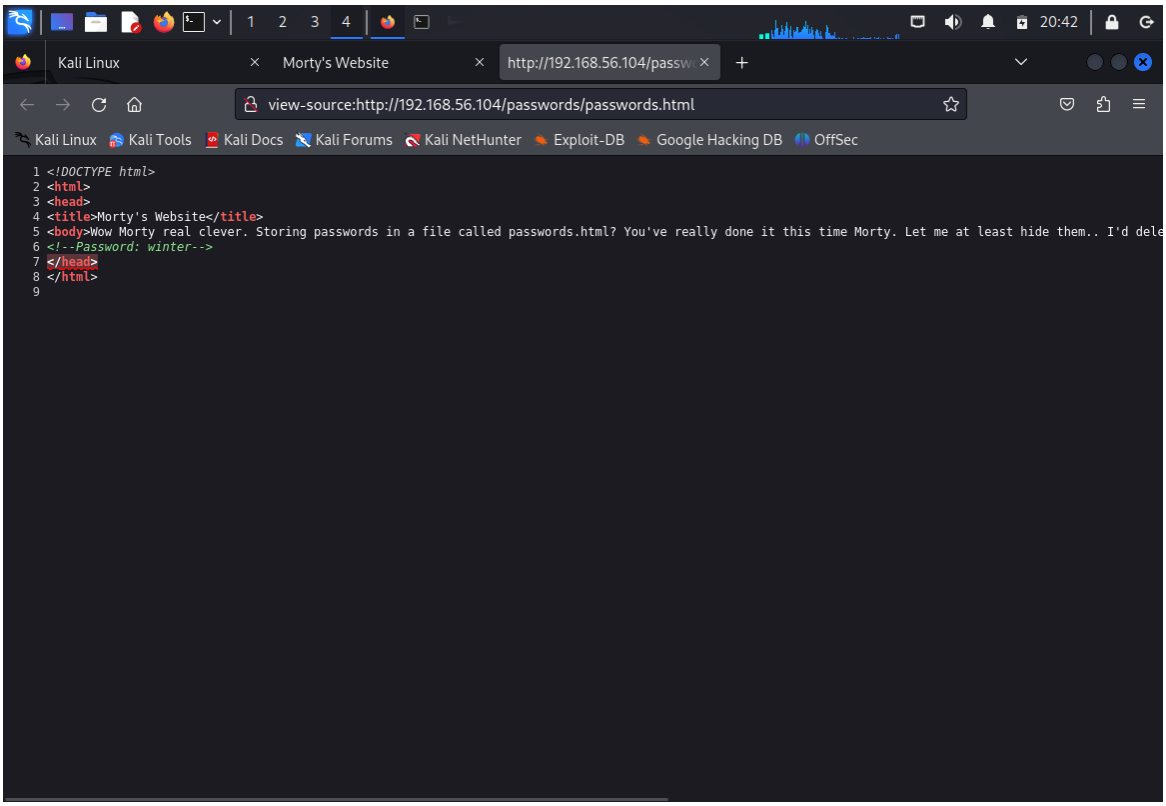
MORTY'S MACHINE TRACER MACHINE

Enter an IP address to trace.

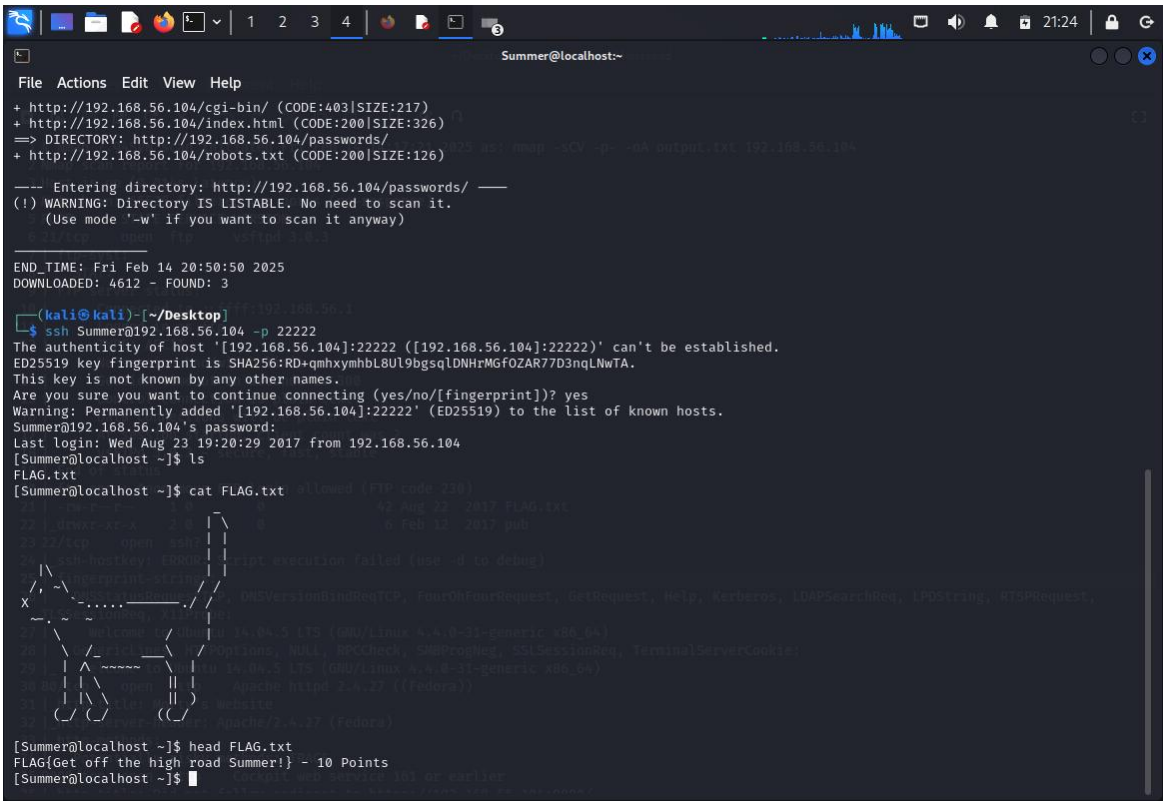
Trace!

```
rpc:x:32:32:Rpcbind Daemon:/var/lib/rpcbind:/sbin/nologin
abrt:x:173:173::/etc/abrt:/sbin/nologin
cockpit-ws:x:996:994:User for cockpit-ws:/:/sbin/nologin
rpcuser:x:29:29:RPC Service User:/var/lib/nfs:/sbin/nologin
chrony:x:995:993::/var/lib/chrony:/sbin/nologin
tcpdump:x:72:72::/:/sbin/nologin
RickSanchez:x:1000:1000::/home/RickSanchez:/bin/bash
Morty:x:1001:1001::/home/Morty:/bin/bash
Summer:x:1002:1002::/home/Summer:/bin/bash
apache:x:48:48:Apache:/usr/share/httpd:/sbin/nologin
```

Reading /etc/passwd reveals three unique user accounts.



The credential "winter" found in an HTML comment in passwords.html.



SSH login as Summer (port 22222) succeeds and the flag is read.

FLAG **{Get off the high road Summer!}** — *10 Points*

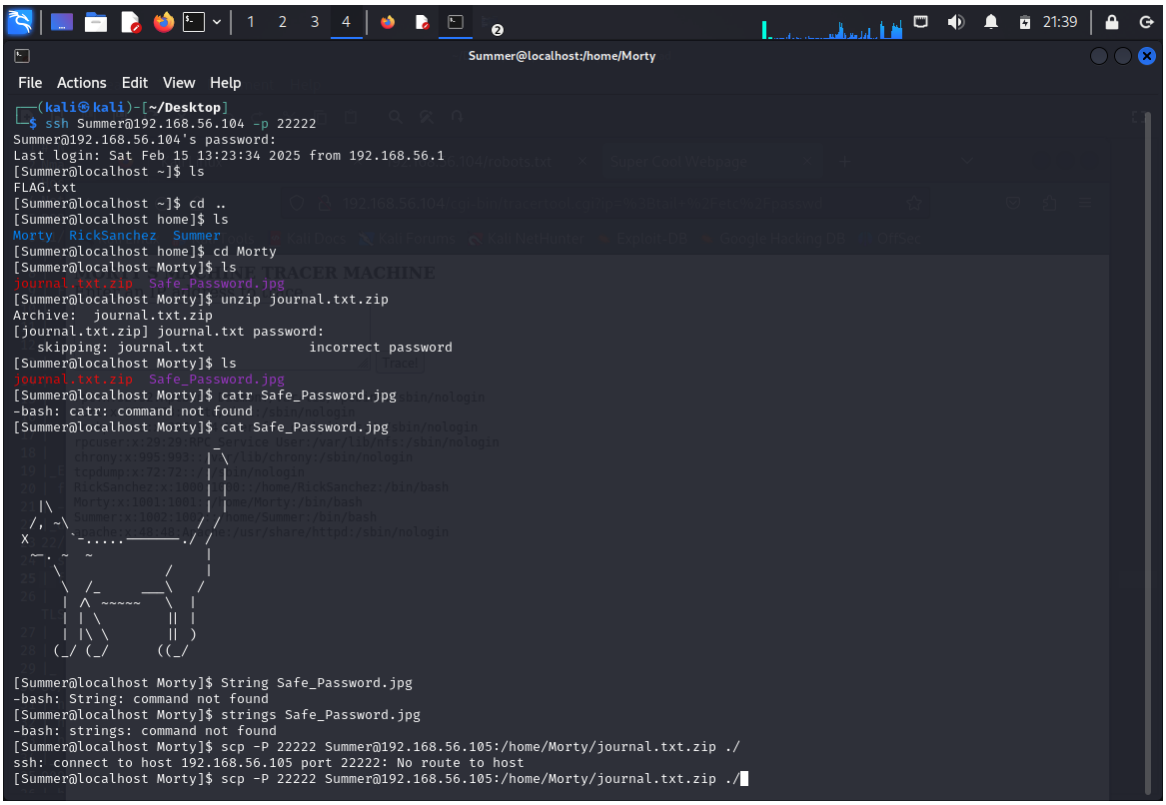
Flag 7 — Protected Archive & Simple Steganography (20 Points)

Finding

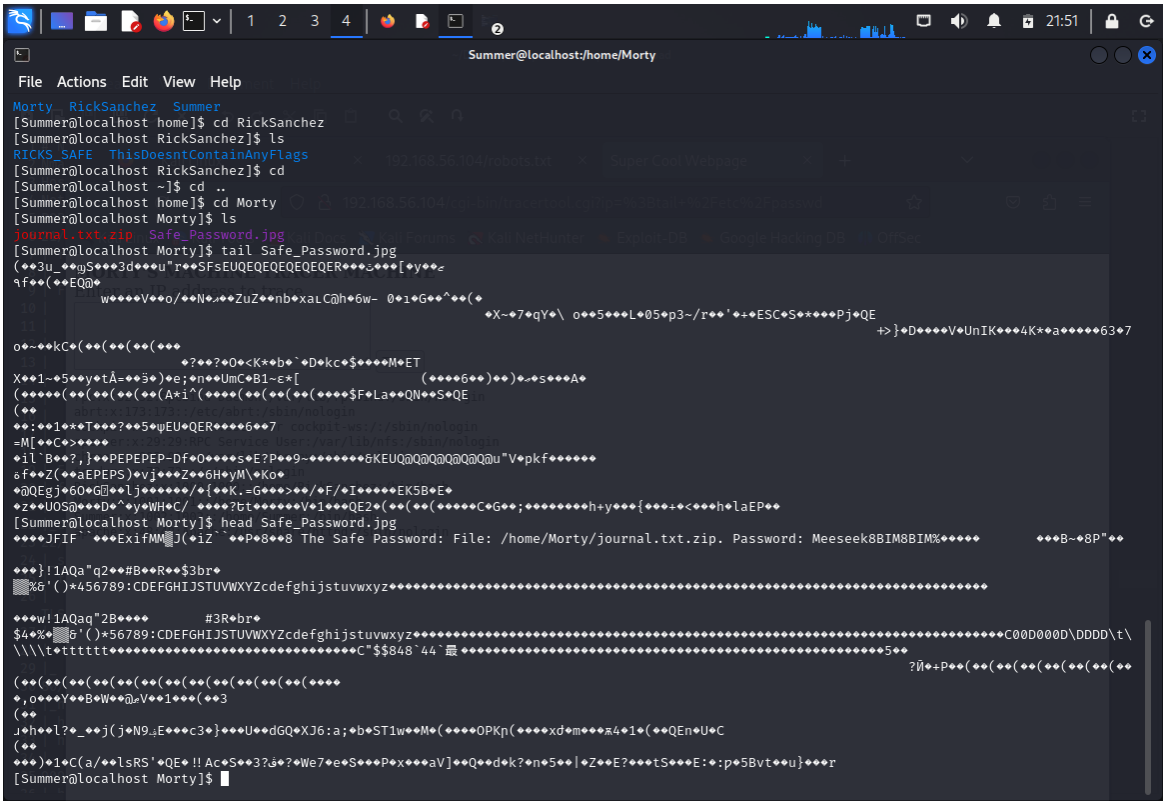
Inside Morty's home directory, a password-protected archive journal.txt.zip was found, along with an image file Safe_Password.jpg containing a hidden password at the end of the file/in its metadata.

Steps Taken

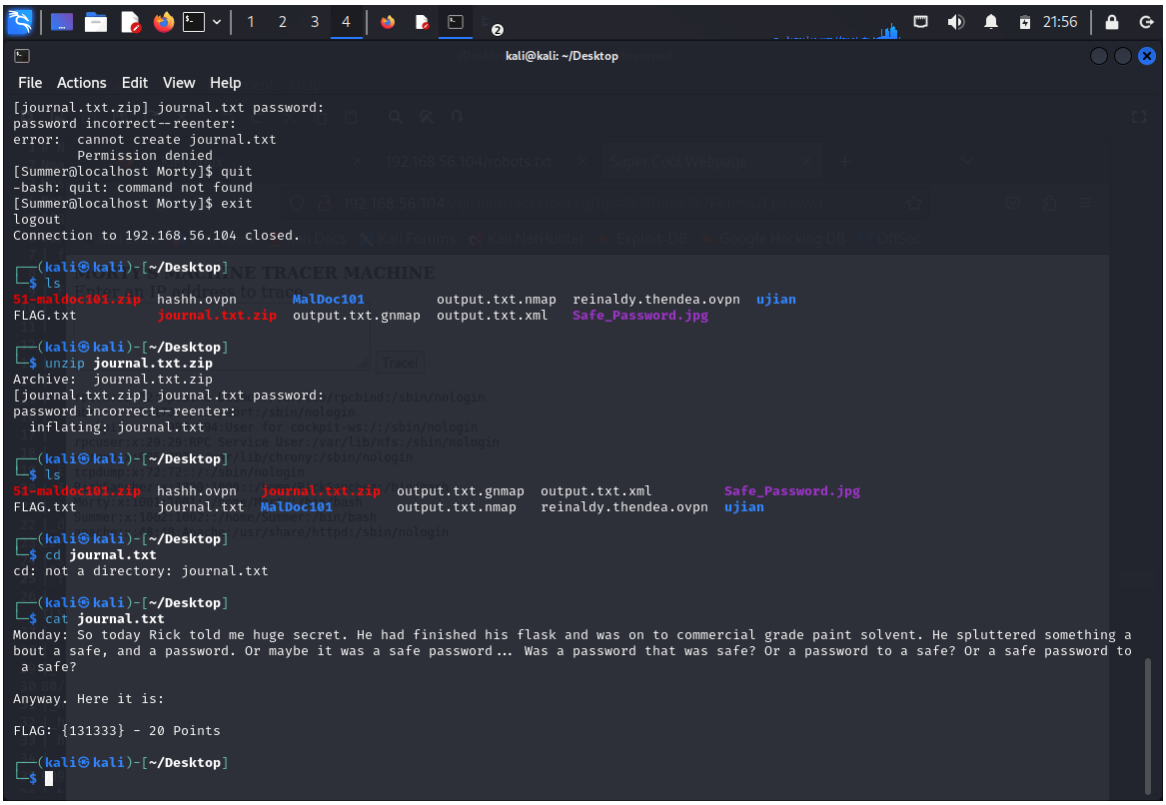
- As user Summer, explored another user's home directory: cd /home/Morty.
- Found two files of interest: journal.txt.zip and Safe_Password.jpg. Attempting to unzip journal.txt.zip prompted for a password and failed.
- Examined Safe_Password.jpg with head and tail — text containing the archive password was found near the end of the file.
- Copied journal.txt.zip to the attacking machine using scp -P 22222 Summer@192.168.56.104:/home/Morty/journal.txt.zip ./, then unzipped it using the password found.
- The contents of journal.txt were a short note from Morty, ending with the flag.



Exploring Morty's directory; the initial unzip attempt fails due to the password-protected archive.



Examining Safe_Password.jpg with head/tail reveals the archive password.



The archive is successfully unzipped; journal.txt contains the flag.

FLAG {131333} — *20 Points*

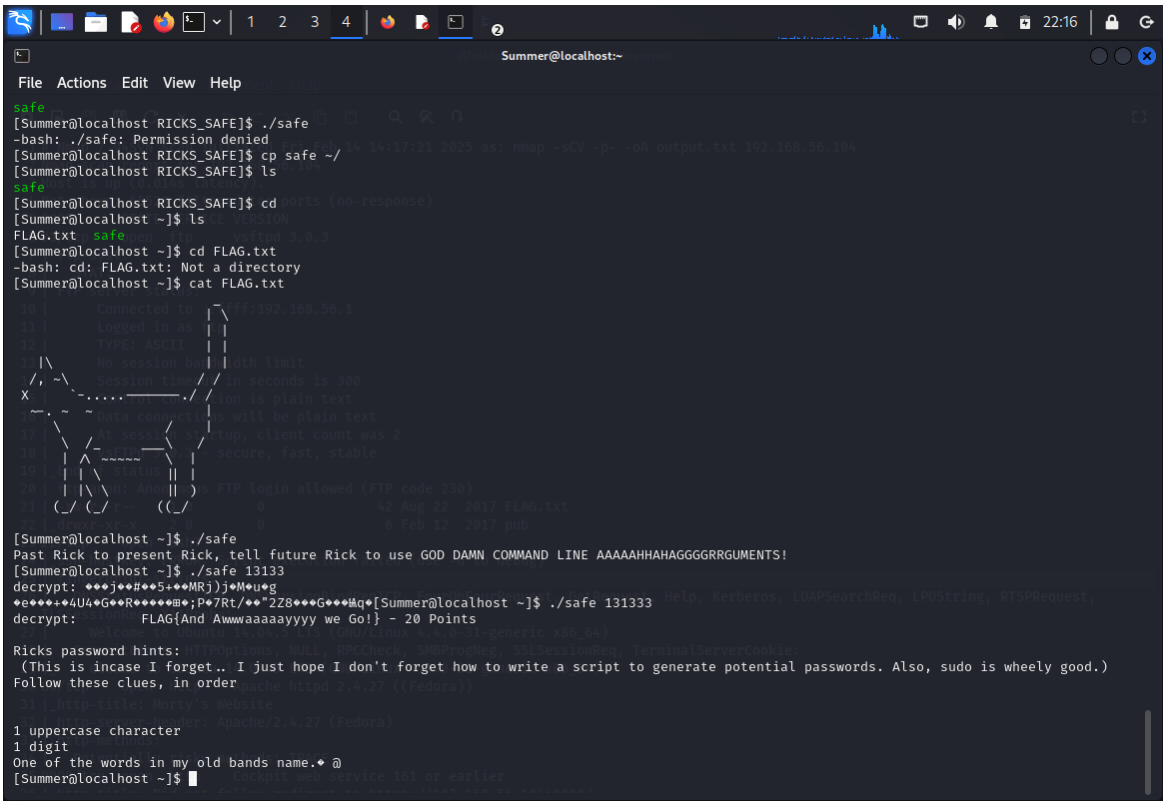
Flag 8 — Executing the “safe” Binary (20 Points)

Finding

In RickSanchez's RICKS_SAFE directory, a binary named safe was found that accepts a numeric password argument to perform a decryption process.

Steps Taken

- Copied the safe binary to the home directory to make it executable (cp safe ~/).
- Ran the binary with the flag value found earlier as its argument: ./safe 131333.
- The decryption succeeded and displayed the flag, along with a hint about the password rules for the next stage (containing 1 uppercase letter, 1 digit, and one word from the name of Rick's old band).



Running ./safe 131333 produces the flag along with a hint for the next password.

FLAG **{And Awwwaaaaayyyy we Go!}** — *20 Points*

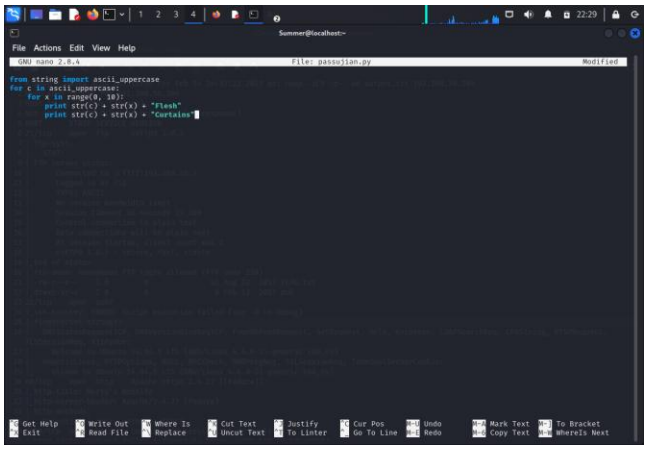
Flag 9 — SSH Brute Force & Privilege Escalation to Root (30 Points)

Finding

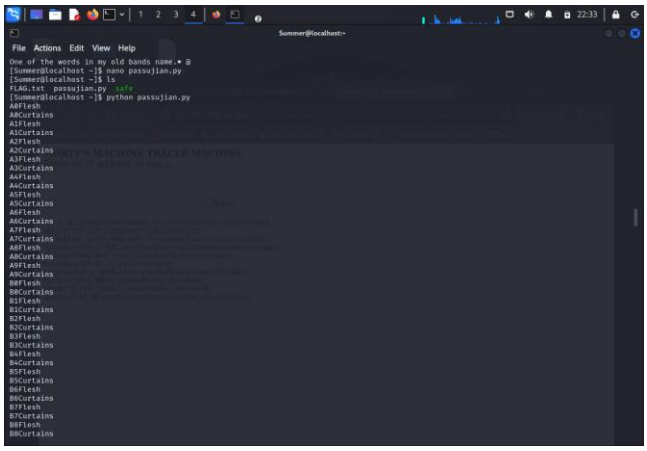
Based on the hint obtained from Flag 8, RickSanchez's account password is made up of a combination of 1 uppercase letter, 1 digit, and the name of Rick's old band — “The Flesh Curtains” (found via a quick search on the RickSanchez character). This pattern could be turned into a custom wordlist for an SSH brute-force attack.

Steps Taken

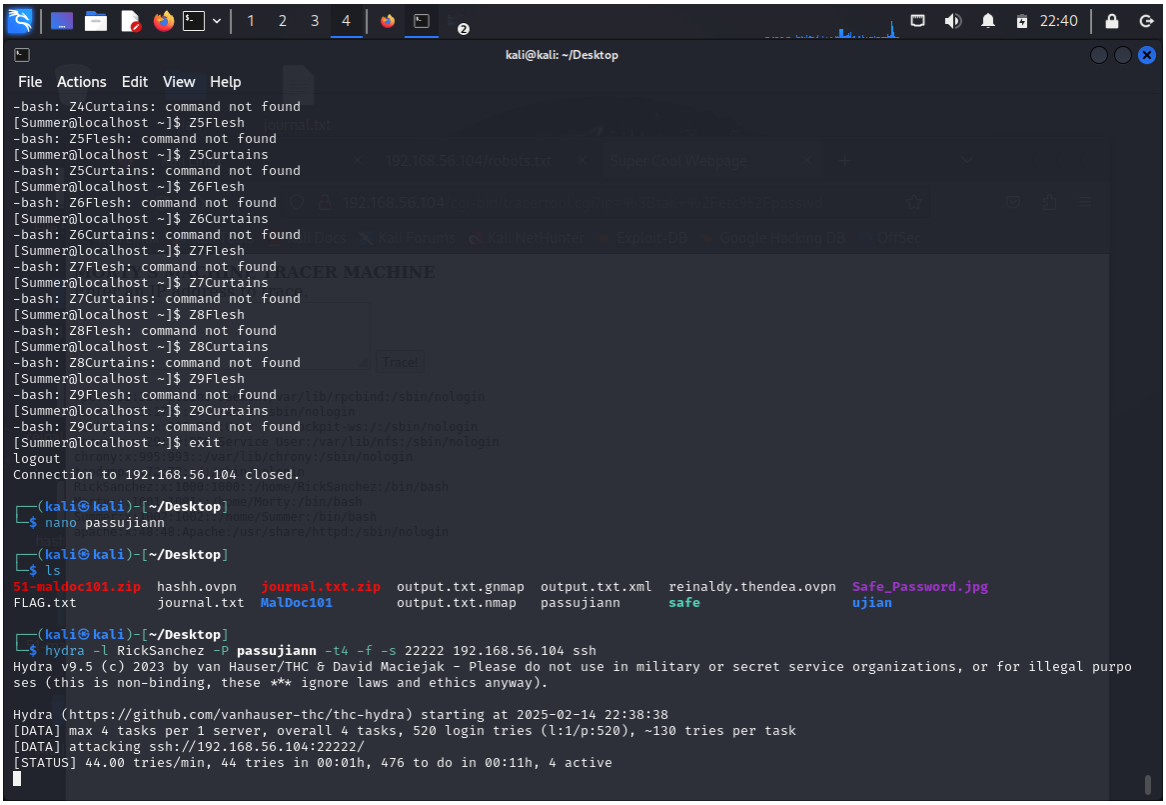
- Wrote a Python script to generate every combination of [A-Z][0-9] + “Flesh” / “Curtains”.
- Ran the script and saved all the generated combinations into a wordlist (passujiann).
- Brute-forced SSH using Hydra: hydra -l RickSanchez -P passujiann -t4 -f -s 22222 192.168.56.104 ssh.
- A valid password was found: RickSanchez : P7Curtains.
- Logged in via SSH as RickSanchez, then ran sudo su to escalate to root.
- As root, the final flag was found and read using head FLAG.txt.



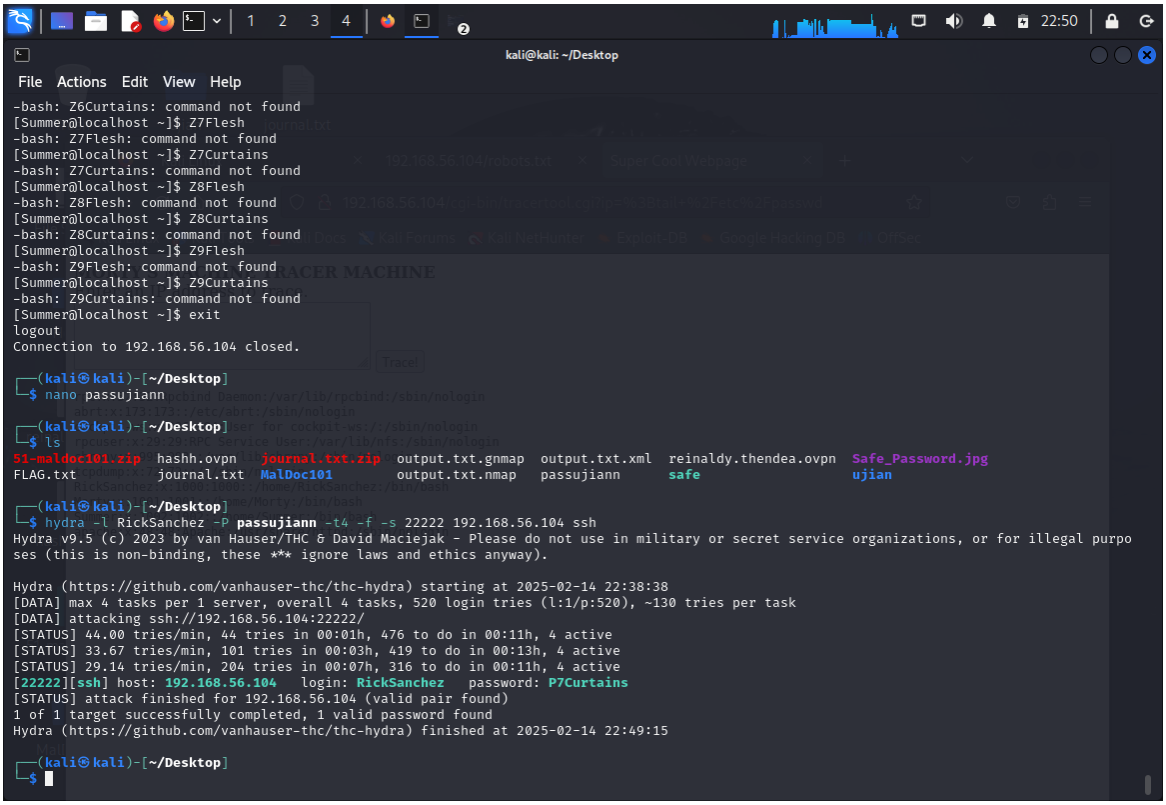
Python script used to generate the wordlist of letter, digit, and band-name combinations.



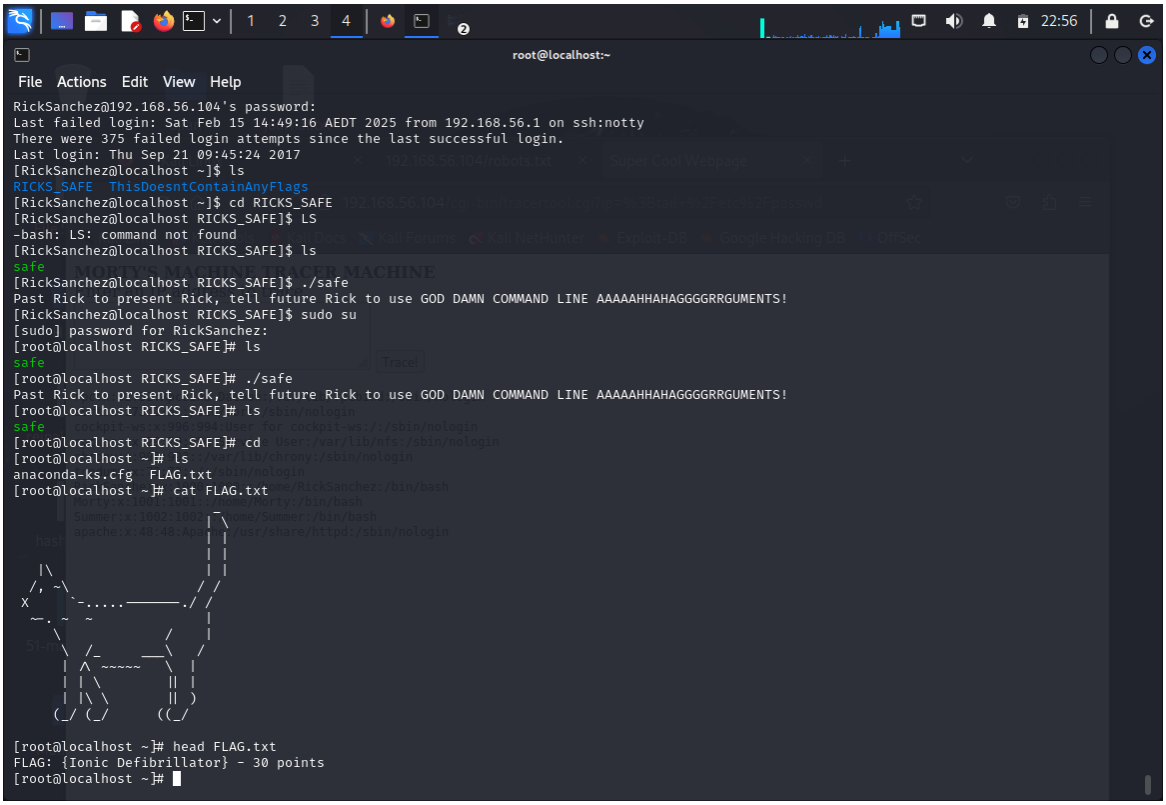
The password combination wordlist is successfully generated.



SSH brute-force attack launched using Hydra.



Hydra successfully finds a valid credential: RickSanchez / P7Curtains.



Privilege escalation to root via sudo su; the final flag is successfully read.

FLAG `{Ionic Defibrillator}` — *30 Points*

Conclusion

The RickdiculouslyEasy lab was fully completed, capturing all 9 out of 9 flags (130 points total). This exercise combined service enumeration, exploitation of a web application command injection vulnerability, analysis of insecurely stored credentials (hardcoded/HTML comment), extraction of hidden information from media files, and brute-force plus privilege escalation to root access.

Several root-cause security issues were identified during this process, including: an FTP service that allows anonymous access to sensitive files, active directory listing on a directory containing confidential data, unsanitized user input in a CGI script leading to command injection, credentials stored in HTML comments/image metadata, and a weak password policy that could be guessed through simple reconnaissance (band name, letter-digit patterns).